

Module 7

ERC Standards (ERC-20, ERC-721)

What Is an ERC?

ERC = Ethereum Request for Comments

Real-World Analogy:

USB is a standard for chargers

ERCs are standards for Ethereum

Technical Definition:

ERC defines interface & behavior

All compliant contracts expose same functions

Enables wallets, exchanges, & DeFi

Without Standards:

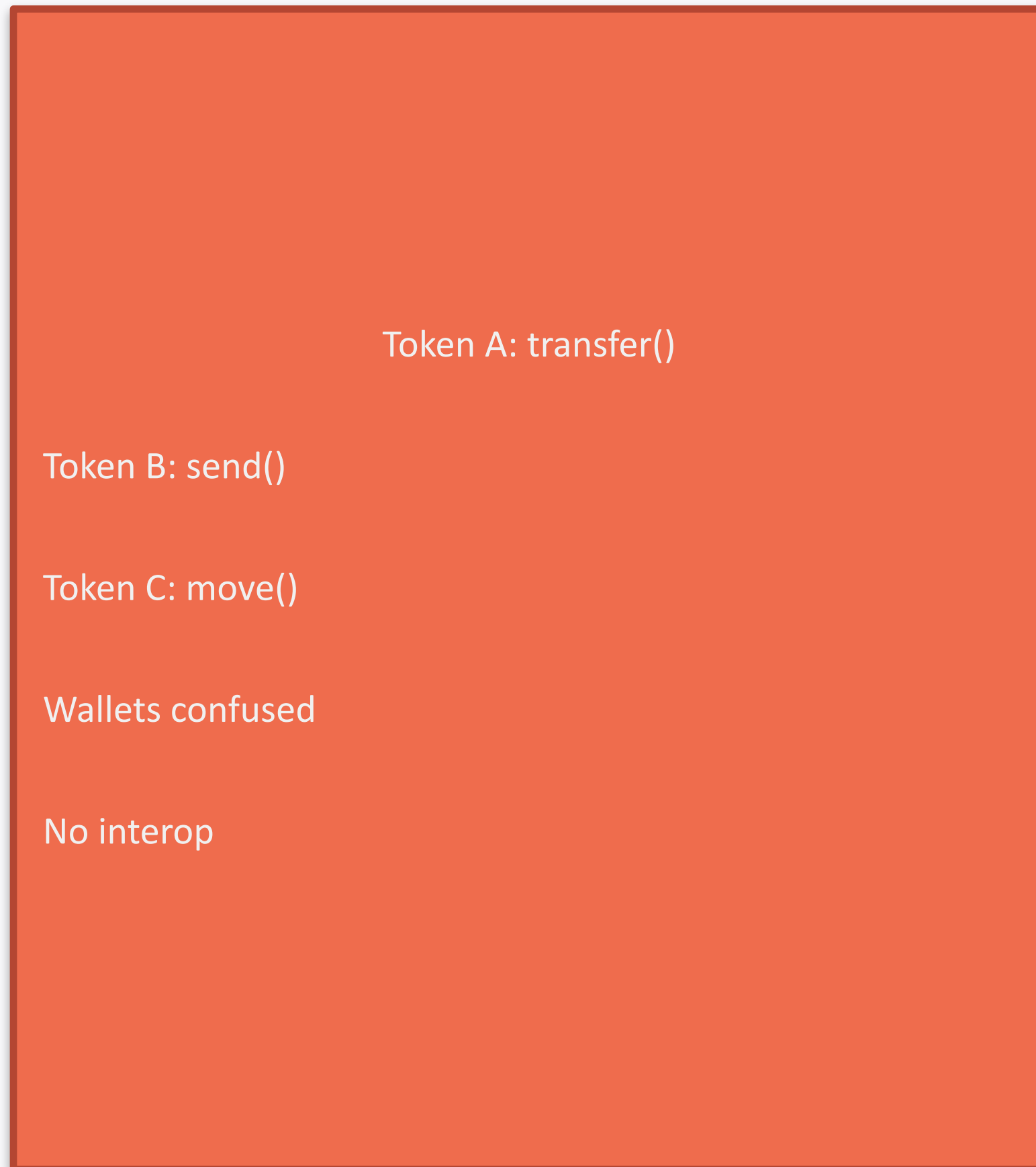
Every token = different functions

No interoperability

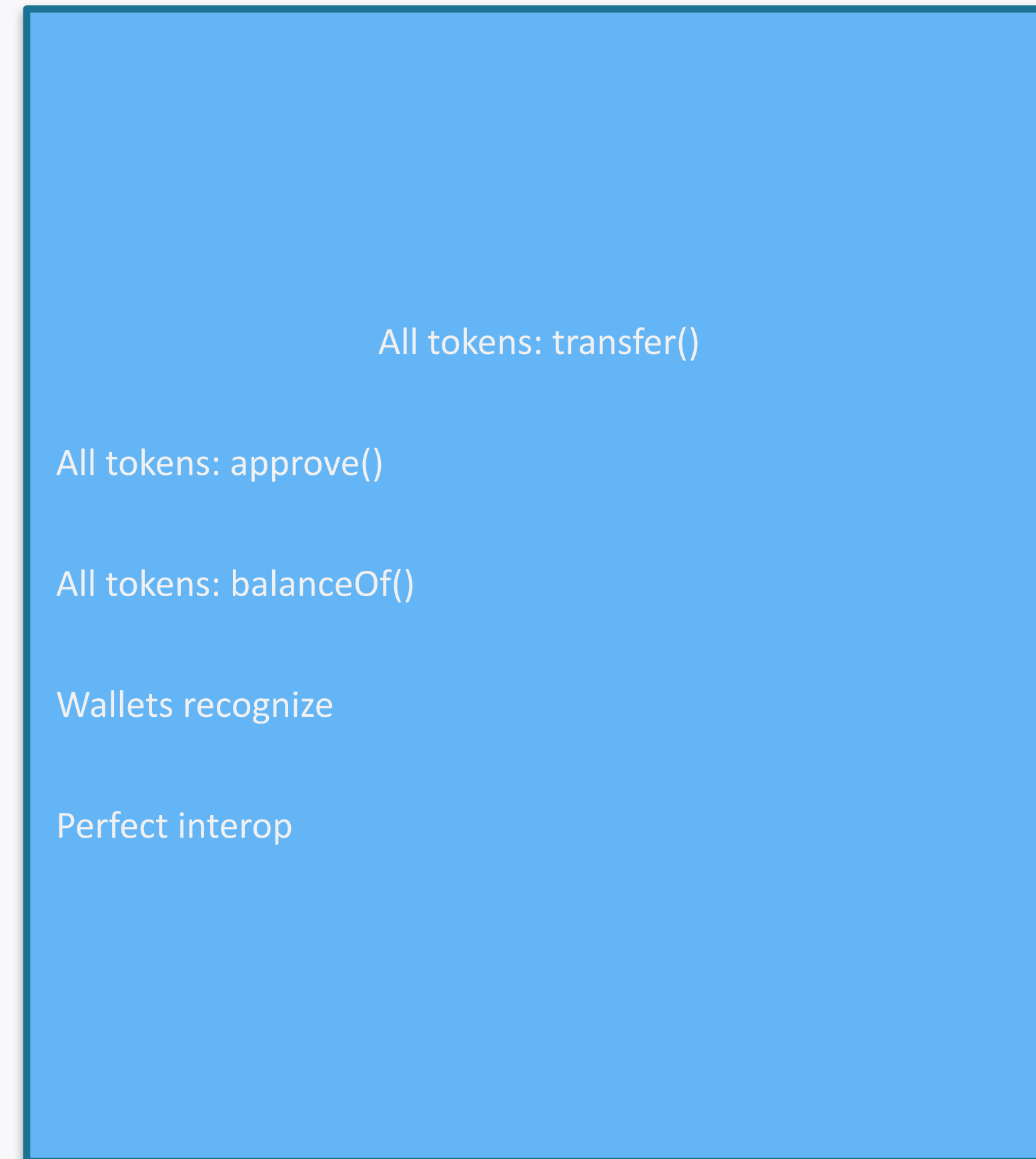
No ecosystem

Standards Enable Interoperability

Without Standards



With ERC-20 Standard



Major ERC Standards

ERC-20

Fungible Tokens

Identical tokens like dollars

ERC-721

Non-Fungible (NFTs)

Unique tokens like deeds

ERC-1155

Multi-Token

Mix of fungible + non-fungible

ERC-20 Interface: Core Functions

```
function totalSupply()
    external view returns(uint);

function balanceOf(address account)
    external view returns(uint);

function transfer(address to,
    uint256 amount)
    external returns(bool);

function approve(address spender,
    uint256 amount)
    external returns(bool);

function transferFrom(
    address from, address to,
    uint256 amount)
    ) external returns(bool);
```

6 core functions

Read: totalSupply(),
balanceOf(), allowance()

Write: transfer(),
approve(), transferFrom()

ERC-20 Required Events

Event: Transfer

Emitted on every token transfer

emit Transfer(from, to, amount)

Includes mint (from = address(0))

Event: Approval

Emitted when approve() called

emit Approval(owner, spender, amount)

Why Events Matter:

Enable off-chain indexing

Wallets track history

Required by standard

Understanding Allowance (Two-Step Transfer)

Real-World Analogy:

Give someone permission to spend \$50
from your bank account

In Smart Contracts:

`approve(spender, amount)` = permission
`transferFrom()` = actual spending

Why Two Steps?

Safer than direct transfer
Spender doesn't hold keys
Owner retains control

Common Use Case:

DEX swaps, smart contracts

Minimal ERC-20 Implementation

```
contract MyToken {  
    mapping(address => uint)  
        balances;  
    mapping(address =>  
        mapping(address => uint))  
        allowances;  
  
    function transfer(address to,  
        uint amount) public {  
        require(balances[msg.sender]  
            >= amount);  
        balances[msg.sender] -= amount;  
        balances[to] += amount;  
        emit Transfer(msg.sender,  
            to, amount);  
    }  
}
```

Core pattern:

Check balance

Update state

Emit event

Follows CEI pattern

ERC-20 vs ERC-721

ERC-20

Fungible Tokens
Like dollars

balanceOf: amount

transfer(address, uint)

approve & allowance

DeFi, Governance

ERC-721

Non-Fungible (NFTs)
Like house deeds

ownerOf(tokenId)

transfer(address, uint)

tokenURI for metadata

Art, Collectibles

ERC-721 (NFT Standard)

Key Differences from ERC-20:

Each token has unique ID

ownerOf(tokenId) → owner address

balanceOf() → count of tokens

transfer() → specific tokenId

Core Functions:

ownerOf(tokenId)

safeTransferFrom(from, to, tokenId)

approve(to, tokenId)

tokenURI(tokenId) → metadata URL

Use Cases:

Art, collectibles, gaming

Minimal ERC-721 Implementation

```
contract SimpleNFT {  
    mapping(uint => address)  
        public ownerOf;  
    uint nextTokenId;  
  
    event Transfer(address indexed from,  
        address to, uint tokenId);  
  
    function mint() public {  
        uint id = nextTokenId++;  
        ownerOf[id] = msg.sender;  
        emit Transfer(address(0),  
            msg.sender, id);  
    }  
}
```

One token per ID

nextTokenId tracks
unique IDs

Mint creates new
unique token

ERC-1155 (Multi-Token Standard)

The "All-in-One" Standard

Supports both fungible & non-fungible
in single contract

Real-World Analogy:

Warehouse inventory system
Some items identical (fungible)
Some items unique (non-fungible)

Use Cases:

Gaming (weapons, currency)
Complex token systems

Note:

Advanced topic — covered later

Interfaces: Contract Between Developers

What Is an Interface?

Blueprint of required functions

"If you implement these, I can use you"

Solidity Example:

```
interface IERC20 {  
    function transfer(address to,  
        uint amount) external;  
}
```

Why They Matter:

Enables composability

Wallets interact with ANY ERC-20

via same interface

Standards Create the Ecosystem

Without Standards:

- Each contract = isolated island
- No wallet support
- No DeFi composability

With ERC-20 Standard:

- Any wallet supports all tokens
- DEXs work with all tokens
- Lending protocols compatible

Result:

- Entire DeFi ecosystem enabled
- One standard changed crypto

Common ERC-20 Mistakes

Allowance Issues:

- Not decreasing allowance
- Allowing overflow/underflow
- Not emitting Approval event

Transfer Issues:

- Forgetting zero address check
- Not emitting Transfer event
- Missing balance validation

Security Issues:

- Reentrancy in complex logic
- Allowance race conditions

Module 7 Recap: ERC Standards

- ✓ ERCs are standards, like USB
- ✓ ERC-20 = Fungible tokens (dollars)
- ✓ ERC-721 = NFTs (unique deeds)
- ✓ Allowance enables safe spending
- ✓ Events required for indexing
- ✓ Interfaces enable interoperability

Key Insight:

Standards transform isolated contracts
into a composable ecosystem

Without ERC-20: No DeFi

Standards enabled everything